

JAVA

TP-Cours

Programmation Orientée Objet

Table des Matières

Introduction

Partie 1 : Cours Intégré - Les Bases de la POO

1.1 Classe et Objet

1.2 Attributs (Variables d'instance)

1.3 Méthodes

1.4 Getters et Setters

1.5 Constructeur

1.6 Les Quatre Piliers de la POO

Partie 2 : TP - Système de Gestion de Bibliothèque

2.1 Objectifs du TP

2.2 Exercice 1 : Classe Abstraite Document

2.3 Exercice 2 : Classes Dérivées

2.4 Exercice 3 : Classe Membre

2.5 Exercice 4 : Classe Emprunt

2.6 Exercice 5 : Classe Bibliothèque

2.7 Exercice 6 : Programme Principal

Partie 3 : Questions

Partie 4 : Exercices

Introduction

Bienvenue dans ce TP complet sur la Programmation Orientée Objet en Java. Ce document combine à la fois un cours théorique et des exercices pratiques détaillés. Vous allez créer un système complet de gestion de bibliothèque qui vous permettra de maîtriser les concepts fondamentaux de la POO : classes, objets, encapsulation, héritage, polymorphisme et abstraction.

Prérequis : Connaissances de base en programmation (variables, conditions, boucles)

Partie 1 : Cours Intégré - Les Bases de la POO

1.1 Classe et Objet

Une classe est un modèle (blueprint) qui définit les caractéristiques et les comportements d'un type d'objet. C'est comme un plan architectural pour construire des maisons.

Un objet est une instance concrète d'une classe. C'est comme une maison construite à partir d'un plan architectural.

Exemple :

```
public class Livre {  
    // Ceci est une classe (le modèle)  
}
```

```
Livre monLivre = new Livre(); // Ceci est un objet (l'instance)
```

□ Analogie : La classe Livre est comme le concept de "livre" en général. L'objet monLivre est un livre spécifique, comme "Le Petit Prince".

1.2 Attributs (Variables d'instance)

Les attributs représentent l'état d'un objet, c'est-à-dire ses données. Chaque objet possède ses propres valeurs d'attributs.

Exemple :

```
public class Livre {  
    private String titre;  
    private String auteur;  
    private double prix;  
    private int anneePublication;  
}
```

□□ Important : Les attributs sont généralement déclarés private pour respecter le principe d'encapsulation (voir section 1.6.2).

1.3 Méthodes

Les méthodes définissent les comportements d'un objet, c'est-à-dire les actions qu'il peut effectuer.

Exemple :

```
public class Livre {
    private String titre;
    private double prix;

    // Méthode pour afficher les informations
    public void afficher() {
        System.out.println("Titre: " + titre);
        System.out.println("Prix: " + prix + "€");
    }

    // Méthode pour appliquer une réduction
    public void appliquerReduction(double pourcentage) {
        prix = prix * (1 - pourcentage / 100);
    }
}
```

1.4 Getters et Setters

Les getters et setters sont des méthodes qui permettent d'accéder et de modifier les attributs privés d'une classe de manière contrôlée.

Un getter (accesseur) : permet de lire la valeur d'un attribut

Un setter (mutateur) : permet de modifier la valeur d'un attribut

Exemple :

```
public class Livre {
    private String titre;
    private double prix;

    // Getter pour titre
    public String getTitre() {
        return titre;
    }

    // Setter pour titre
    public void setTitre(String titre) {
        this.titre = titre;
    }

    // Getter pour prix
    public double getPrix() {
        return prix;
    }
}
```

```

        // Setter pour prix avec validation
        public void setPrix(double prix) {
            if (prix >= 0) {
                this.prix = prix;
            } else {
                System.out.println("Le prix doit être positif!");
            }
        }
    }
}

```

□ **Avantage :** Les setters permettent d'ajouter de la validation et du contrôle sur les données.

1.5 Constructeur

Un constructeur est une méthode spéciale qui est appelée automatiquement lors de la création d'un objet. Il permet d'initialiser les attributs.

Exemple :

```

public class Livre {
    private String titre;
    private String auteur;
    private double prix;

    // Constructeur
    public Livre(String titre, String auteur, double prix) {
        this.titre = titre;
        this.auteur = auteur;
        this.prix = prix;
    }
}

// Utilisation
Livre livre1 = new Livre("1984", "George Orwell", 15.99);

```

□ **Note :** Le mot-clé "this" fait référence à l'objet courant.

1.6 Les Quatre Piliers de la POO

1.6.1 Abstraction

L'abstraction consiste à masquer les détails complexes et à ne montrer que l'essentiel. En Java, on utilise des classes abstraites et des interfaces.

Une classe abstraite est une classe qui ne peut pas être instanciée directement. Elle sert de modèle pour d'autres classes.

Exemple :

```

public abstract class Document {
    private String titre;

```

```

// Méthode abstraite (sans implémentation)
public abstract void afficher();

// Méthode concrète (avec implémentation)
public String getTitre() {
    return titre;
}
}

```

1.6.2 Encapsulation

L'encapsulation est le principe de cacher les données internes d'un objet et de ne les rendre accessibles que via des méthodes publiques (getters/setters).

□ Avantages de l'encapsulation :

- • Protection des données contre les modifications non contrôlées
- • Flexibilité pour modifier l'implémentation interne
- • Validation des données via les setters

Exemple :

```

public class CompteBancaire {
    private double solde; // private = encapsulé

    public double getSolde() {
        return solde;
    }

    public void deposer(double montant) {
        if (montant > 0) {
            solde += montant;
        }
    }

    public void retirer(double montant) {
        if (montant > 0 && montant <= solde) {
            solde -= montant;
        }
    }
}

```

1.6.3 Héritage

L'héritage permet à une classe (classe fille ou sous-classe) de hériter des attributs et méthodes d'une autre classe (classe parent ou super-classe).

□ Avantages de l'héritage :

- • Réutilisation du code
- • Organisation hiérarchique des classes

- • Facilite la maintenance

Exemple :

```
// Classe parent
public class Document {
    protected String titre;
    protected int annee;

    public void afficherInfosBase() {
        System.out.println("Titre: " + titre);
    }
}

// Classe fille
public class Livre extends Document {
    private String auteur;

    public void afficherInfos() {
        afficherInfosBase(); // Appel méthode parent
        System.out.println("Auteur: " + auteur);
    }
}
```

□ Note : Le mot-clé "extends" indique l'héritage. Le mot-clé "super" permet d'accéder aux membres de la classe parent.

1.6.4 Polymorphisme

Le polymorphisme signifie "plusieurs formes". Il permet à des objets de types différents d'être traités de manière uniforme. Une même méthode peut avoir des comportements différents selon l'objet qui l'appelle.

Exemple :

```
public abstract class Document {
    public abstract void afficher();
}

public class Livre extends Document {
    @Override
    public void afficher() {
        System.out.println("Affichage d'un livre");
    }
}

public class Magazine extends Document {
    @Override
    public void afficher() {
        System.out.println("Affichage d'un magazine");
    }
}
```

```
}
```

```
// Polymorphisme en action  
Document doc1 = new Livre();  
Document doc2 = new Magazine();  
doc1.afficher(); // "Affichage d'un livre"  
doc2.afficher(); // "Affichage d'un magazine"
```

□ **Avantage** : Le polymorphisme permet d'écrire du code flexible et extensible.

Partie 2 : TP - Système de Gestion de Bibliothèque

2.1 Objectifs du TP

Dans ce TP, vous allez créer un système complet de gestion de bibliothèque qui permettra de :

- • Gérer différents types de documents (livres, magazines, DVD)
- • Gérer les membres de la bibliothèque
- • Gérer les emprunts et retours de documents
- • Calculer automatiquement les amendes pour les retards
- • Afficher des statistiques sur la bibliothèque

□ Compétences développées :

- • Création et utilisation de classes abstraites
- • Héritage et polymorphisme
- • Encapsulation avec getters/setters
- • Gestion de collections (ArrayList)
- • Manipulation de dates en Java

2.2 Exercice 1 : Classe Abstraite Document

Objectif : Créer une classe abstraite Document qui servira de base pour tous les types de documents de la bibliothèque.

Instructions :

1. Créez une classe abstraite nommée "Document"
2. Ajoutez les attributs privés suivants :
 - - id (String) : identifiant unique du document
 - - titre (String) : titre du document
 - - anneePublication (int) : année de publication
 - - estEmprunte (boolean) : état d'emprunt du document
3. Créez un constructeur qui initialise tous les attributs sauf estEmprunte (qui doit être false par défaut)
4. Créez les getters et setters pour tous les attributs
5. Créez les méthodes abstraites suivantes :
 - - getType() : retourne le type de document (String)
 - - getDureeEmpruntMax() : retourne la durée maximale d'emprunt en jours (int)
6. Créez une méthode concrète afficherInfos() qui affiche les informations du document

Solution :

```
public abstract class Document {
    // Attributs privés (ENCAPSULATION)
    private String id;
    private String titre;
    private int anneePublication;
    private boolean estEmprunte;

    // Constructeur
    public Document(String id, String titre, int anneePublication) {
        this.id = id;
        this.titre = titre;
        this.anneePublication = anneePublication;
        this.estEmprunte = false;
    }

    // Getters et Setters
    public String getId() {
        return id;
    }

    public String getTitre() {
        return titre;
    }
}
```

```

    }

    public void setTitre(String titre) {
        this.titre = titre;
    }

    public int getAnneePublication() {
        return anneePublication;
    }

    public void setAnneePublication(int anneePublication) {
        this.anneePublication = anneePublication;
    }

    public boolean isEstEmprunte() {
        return estEmprunte;
    }

    public void setEstEmprunte(boolean estEmprunte) {
        this.estEmprunte = estEmprunte;
    }

    // Méthodes abstraites (ABSTRACTION)
    public abstract String getType();
    public abstract int getDureeEmpruntMax();

    // Méthode concrète
    public void afficherInfos() {
        System.out.println("=== Informations du Document ===");
        System.out.println("ID: " + id);
        System.out.println("Titre: " + titre);
        System.out.println("Année: " + anneePublication);
        System.out.println("Type: " + getType());
        System.out.println("Statut: " + (estEmprunte ? "Emprunté" :
"Disponible"));
    }
}

```

Questions de compréhension :

1. Pourquoi les attributs sont-ils déclarés private ?
2. Pourquoi la classe Document est-elle abstraite ?
3. Quelle est la différence entre une méthode abstraite et une méthode concrète ?
4. Pourquoi utilise-t-on "this" dans le constructeur ?

2.3 Exercice 2 : Classes Dérivées (Héritage et Polymorphisme)

2.3.1 Classe Livre

Objectif : Créer une classe Livre qui hérite de Document.

Instructions :

1. Créez une classe "Livre" qui extends Document
2. Ajoutez les attributs privés supplémentaires :
 - - auteur (String)
 - - isbn (String)
 - - nombrePages (int)
3. Créez un constructeur qui prend tous les paramètres (y compris ceux du parent)
4. Créez les getters et setters pour les nouveaux attributs
5. Implémentez les méthodes abstraites :
 - - getType() doit retourner "Livre"
 - - getDureeEmpruntMax() doit retourner 21 (jours)
6. Redéfinissez la méthode afficherInfos() pour inclure les informations spécifiques au livre

Solution :

```
public class Livre extends Document {
    private String auteur;
    private String isbn;
    private int nombrePages;

    // Constructeur
    public Livre(String id, String titre, int anneePublication,
                 String auteur, String isbn, int nombrePages) {
        super(id, titre, anneePublication); // Appel au constructeur
        parent
        this.auteur = auteur;
        this.isbn = isbn;
        this.nombrePages = nombrePages;
    }

    // Getters et Setters
    public String getAuteur() {
        return auteur;
    }

    public void setAuteur(String auteur) {
        this.auteur = auteur;
    }
}
```

```

public String getIsbn() {
    return isbn;
}

public void setIsbn(String isbn) {
    this.isbn = isbn;
}

public int getNombrePages() {
    return nombrePages;
}

public void setNombrePages(int nombrePages) {
    this.nombrePages = nombrePages;
}

// Implémentation des méthodes abstraites (POLYMORPHISME)
@Override
public String getType() {
    return "Livre";
}

@Override
public int getDureeEmpruntMax() {
    return 21; // 21 jours pour un livre
}

// Redéfinition de la méthode afficherInfos (POLYMORPHISME)
@Override
public void afficherInfos() {
    super.afficherInfos(); // Appel à la méthode parent
    System.out.println("Auteur: " + auteur);
    System.out.println("ISBN: " + isbn);
    System.out.println("Nombre de pages: " + nombrePages);
    System.out.println("Durée d'emprunt max: " +
getDureeEmpruntMax() + " jours");
}
}

```

2.3.2 Classe Magazine

À FAIRE : Créez une classe Magazine similaire à Livre avec les caractéristiques suivantes :

- - Hérite de Document
- - Attributs supplémentaires : numeroEdition (int), mois (String)
- - getType() retourne "Magazine"
- - getDureeEmpruntMax() retourne 7 jours

```

public class Magazine extends Document {
    private int numeroEdition;
    private String mois;

    public Magazine(String id, String titre, int anneePublication,
                    int numeroEdition, String mois) {
        super(id, titre, anneePublication);
        this.numeroEdition = numeroEdition;
        this.mois = mois;
    }

    public int getNumeroEdition() {
        return numeroEdition;
    }

    public void setNumeroEdition(int numeroEdition) {
        this.numeroEdition = numeroEdition;
    }

    public String getMois() {
        return mois;
    }

    public void setMois(String mois) {
        this.mois = mois;
    }

    @Override
    public String getType() {
        return "Magazine";
    }

    @Override
    public int getDureeEmpruntMax() {
        return 7; // 7 jours pour un magazine
    }

    @Override
    public void afficherInfos() {
        super.afficherInfos();
        System.out.println("Numéro d'édition: " + numeroEdition);
        System.out.println("Mois: " + mois);
        System.out.println("Durée d'emprunt max: " +
getDureeEmpruntMax() + " jours");
    }
}

```

2.3.3 Classe DVD

À FAIRE : Créez une classe DVD avec les caractéristiques suivantes :

- - Hérite de Document
- - Attributs supplémentaires : réalisateur (String), dureeMinutes (int)
- - getType() retourne "DVD"
- - getDureeEmpruntMax() retourne 14 jours

```
public class DVD extends Document {
    private String realisateur;
    private int dureeMinutes;

    public DVD(String id, String titre, int anneePublication,
               String realisateur, int dureeMinutes) {
        super(id, titre, anneePublication);
        this.realisateur = realisateur;
        this.dureeMinutes = dureeMinutes;
    }

    public String getRealisateur() {
        return realisateur;
    }

    public void setRealisateur(String realisateur) {
        this.realisateur = realisateur;
    }

    public int getDureeMinutes() {
        return dureeMinutes;
    }

    public void setDureeMinutes(int dureeMinutes) {
        this.dureeMinutes = dureeMinutes;
    }

    @Override
    public String getType() {
        return "DVD";
    }

    @Override
    public int getDureeEmpruntMax() {
        return 14; // 14 jours pour un DVD
    }

    @Override
    public void afficherInfos() {
        super.afficherInfos();
    }
}
```

```
        System.out.println("Réalisateur: " + realisateur);
        System.out.println("Durée: " + dureeMinutes + " minutes");
        System.out.println("Durée d'emprunt max: " +
getDureeEmpruntMax() + " jours");
    }
}
```

2.4 Exercice 3 : Classe Membre

Objectif : Créer une classe pour gérer les membres de la bibliothèque.

Instructions :

1. Créez une classe "Membre" avec les attributs privés :

- - idMembre (String)
- - nom (String)
- - prenom (String)
- - email (String)
- - telephone (String)
- - empruntsEnCours (ArrayList<Emprunt>)

2. Créez un constructeur et les getters/setters

3. Créez les méthodes suivantes :

- - ajouterEmprunt(Emprunt e) : ajoute un emprunt à la liste
- - retirerEmprunt(Emprunt e) : retire un emprunt de la liste
- - getNombreEmpruntsEnCours() : retourne le nombre d'emprunts en cours
- - peutEmprunter() : retourne true si le membre a moins de 5 emprunts
- - afficherInfos() : affiche les informations du membre

Solution :

```
import java.util.ArrayList;

public class Membre {
    private String idMembre;
    private String nom;
    private String prenom;
    private String email;
    private String telephone;
    private ArrayList<Emprunt> empruntsEnCours;

    // Constructeur
    public Membre(String idMembre, String nom, String prenom,
                  String email, String telephone) {
        this.idMembre = idMembre;
        this.nom = nom;
        this.prenom = prenom;
        this.email = email;
        this.telephone = telephone;
        this.empruntsEnCours = new ArrayList<>();
    }

    // Getters et Setters
    public String getIdMembre() {
        return idMembre;
    }
}
```

```

    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getPrenom() {
        return prenom;
    }

    public void setPrenom(String prenom) {
        this.prenom = prenom;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getTelephone() {
        return telephone;
    }

    public void setTelephone(String telephone) {
        this.telephone = telephone;
    }

    public ArrayList<Emprunt> getEmpruntsEnCours() {
        return empruntsEnCours;
    }

    // Méthodes métier
    public void ajouterEmprunt(Emprunt emprunt) {
        empruntsEnCours.add(emprunt);
    }

    public void retirerEmprunt(Emprunt emprunt) {
        empruntsEnCours.remove(emprunt);
    }

```

```
public int getNombreEmpruntsEnCours() {
    return empruntsEnCours.size();
}

public boolean peutEmprunter() {
    return empruntsEnCours.size() < 5; // Maximum 5 emprunts
}

public void afficherInfos() {
    System.out.println("=== Informations Membre ===");
    System.out.println("ID: " + idMembre);
    System.out.println("Nom: " + nom + " " + prenom);
    System.out.println("Email: " + email);
    System.out.println("Téléphone: " + telephone);
    System.out.println("Emprunts en cours: " +
empruntsEnCours.size());
}
}
```

2.5 Exercice 4 : Classe Emprunt

Objectif : Créer une classe pour gérer les emprunts de documents.

Instructions :

1. Créez une classe "Emprunt" avec les attributs privés :

- - idEmprunt (String)
- - membre (Membre)
- - document (Document)
- - dateEmprunt (LocalDate)
- - dateRetourPrevu (LocalDate)
- - dateRetourEffectif (LocalDate)
- - amende (double)

2. Le constructeur doit calculer automatiquement dateRetourPrevu en ajoutant la durée d'emprunt au document

3. Créez les méthodes suivantes :

- - estEnRetard() : vérifie si le document est en retard
- - getJoursRetard() : calcule le nombre de jours de retard
- - calculerAmende() : calcule l'amende (0.50€ par jour)
- - effectuerRetour() : enregistre le retour du document

Solution :

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

public class Emprunt {
    private static int compteurId = 1;
    private String idEmprunt;
    private Membre membre;
    private Document document;
    private LocalDate dateEmprunt;
    private LocalDate dateRetourPrevu;
    private LocalDate dateRetourEffectif;
    private double amende;

    // Constructeur
    public Emprunt(Membre membre, Document document) {
        this.idEmprunt = "EMP" + String.format("%04d", compteurId++);
        this.membre = membre;
        this.document = document;
        this.dateEmprunt = LocalDate.now();
        // Calcul automatique de la date de retour prévue
        this.dateRetourPrevu =
            dateEmprunt.plusDays(document.getDureeEmpruntMax());
        this.dateRetourEffectif = null;
    }
}
```

```

        this.amende = 0.0;
    }

    // Getters
    public String getIdEmprunt() {
        return idEmprunt;
    }

    public Membre getMembre() {
        return membre;
    }

    public Document getDocument() {
        return document;
    }

    public LocalDate getDateEmprunt() {
        return dateEmprunt;
    }

    public LocalDate getDateRetourPrevu() {
        return dateRetourPrevu;
    }

    public LocalDate getDateRetourEffectif() {
        return dateRetourEffectif;
    }

    public double getAmende() {
        return amende;
    }

    // Méthodes métier
    public boolean estEnRetard() {
        if (dateRetourEffectif != null) {
            return dateRetourEffectif.isAfter(dateRetourPrevu);
        }
        return LocalDate.now().isAfter(dateRetourPrevu);
    }

    public long getJoursRetard() {
        LocalDate dateRef = (dateRetourEffectif != null) ?
            dateRetourEffectif : LocalDate.now();
        if (dateRef.isAfter(dateRetourPrevu)) {
            return ChronoUnit.DAYS.between(dateRetourPrevu, dateRef);
        }
        return 0;
    }

```

```

    }

    public double calculerAmende() {
        long joursRetard = getJoursRetard();
        if (joursRetard > 0) {
            amende = joursRetard * 0.5; // 0.50€ par jour
        }
        return amende;
    }

    public void effectuerRetour() {
        this.dateRetourEffectif = LocalDate.now();
        calculerAmende();
        document.setEstEmprunte(false);
    }

    public void afficherInfos() {
        System.out.println("== Informations Emprunt ==");
        System.out.println("ID: " + idEmprunt);
        System.out.println("Membre: " + membre.getNom() + " " +
membre.getPrenom());
        System.out.println("Document: " + document.getTitre());
        System.out.println("Date emprunt: " + dateEmprunt);
        System.out.println("Date retour prévue: " + dateRetourPrevu);
        if (dateRetourEffectif != null) {
            System.out.println("Date retour effectif: " +
dateRetourEffectif);
            if (amende > 0) {
                System.out.println("Amende: " + amende + "€");
            }
        } else if (estEnRetard()) {
            System.out.println("RETARD: " + getJoursRetard() + "
jour(s)");
        }
    }
}

```

2.6 Exercice 5 : Classe Bibliothèque

Objectif : Créer la classe principale qui gère toute la bibliothèque.

Instructions :

1. Créez une classe "Bibliotheque" avec les attributs :

- - nom (String)
- - adresse (String)
- - catalogue (ArrayList<Document>)
- - membres (ArrayList<Membre>)
- - emprunts (ArrayList<Emprunt>)

2. Créez les méthodes de gestion :

- - ajouterDocument(Document d)
- - ajouterMembre(Membre m)
- - emprunterDocument(String idMembre, String idDocument)
- - retournerDocument(String idEmprunt)
- - rechercherDocument(String titre)
- - afficherStatistiques()

Solution complète :

```
import java.util.ArrayList;

public class Bibliotheque {
    private String nom;
    private String adresse;
    private ArrayList<Document> catalogue;
    private ArrayList<Membre> membres;
    private ArrayList<Emprunt> emprunts;

    public Bibliotheque(String nom, String adresse) {
        this.nom = nom;
        this.adresse = adresse;
        this.catalogue = new ArrayList<>();
        this.membres = new ArrayList<>();
        this.emprunts = new ArrayList<>();
    }

    // Ajouter un document au catalogue
    public void ajouterDocument(Document document) {
        catalogue.add(document);
        System.out.println("Document ajouté: " + document.getTitre());
    }

    // Ajouter un membre
    public void ajouterMembre(Membre membre) {
```

```

        membres.add(membre);
        System.out.println("Membre ajouté: " + membre.getNom());
    }

    // Rechercher un document par titre
    public Document rechercherDocument(String titre) {
        for (Document doc : catalogue) {
            if (doc.getTitre().equalsIgnoreCase(titre)) {
                return doc;
            }
        }
        return null;
    }

    // Rechercher un membre par ID
    public Membre rechercherMembre(String idMembre) {
        for (Membre m : membres) {
            if (m.getIdMembre().equals(idMembre)) {
                return m;
            }
        }
        return null;
    }

    // Emprunter un document
    public boolean emprunterDocument(String idMembre, String
idDocument) {
        Membre membre = rechercherMembre(idMembre);
        Document document = null;

        // Rechercher le document par ID
        for (Document d : catalogue) {
            if (d.getId().equals(idDocument)) {
                document = d;
                break;
            }
        }

        if (membre == null) {
            System.out.println("Membre non trouvé!");
            return false;
        }

        if (document == null) {
            System.out.println("Document non trouvé!");
            return false;
        }
    }

```

```

        if (document.isEstEmprunte()) {
            System.out.println("Document déjà emprunté!");
            return false;
        }

        if (!membre.peutEmprunter()) {
            System.out.println("Limite d'emprunts atteinte!");
            return false;
        }

        // Créer l'emprunt
        Emprunt emprunt = new Emprunt(membre, document);
        emprunts.add(emprunt);
        membre.ajouterEmprunt(emprunt);
        document.setEstEmprunte(true);

        System.out.println("Emprunt réussi! ID: " +
            emprunt.getIdEmprunt());
        return true;
    }

    // Retourner un document
    public boolean retournerDocument(String idEmprunt) {
        for (Emprunt emp : emprunts) {
            if (emp.getIdEmprunt().equals(idEmprunt)) {
                emp.effectuerRetour();
                emp.getMembre().retirerEmprunt(emp);
                System.out.println("Retour effectué!");
                if (emp.getAmende() > 0) {
                    System.out.println("Amende: " + emp.getAmende() +
                        "€");
                }
                return true;
            }
        }
        System.out.println("Emprunt non trouvé!");
        return false;
    }

    // Afficher les statistiques
    public void afficherStatistiques() {
        System.out.println("\n=== STATISTIQUES ===");
        System.out.println("Bibliothèque: " + nom);
        System.out.println("Documents au catalogue: " +
            catalogue.size());
        System.out.println("Membres inscrits: " + membres.size());
    }

```

```

        System.out.println("Emprunts en cours: " +
compterEmpruntsEnCours());

        // Comptage par type
        int nbLivres = 0, nbMagazines = 0, nbDVDs = 0;
        for (Document d : catalogue) {
            if (d instanceof Livre) nbLivres++;
            else if (d instanceof Magazine) nbMagazines++;
            else if (d instanceof DVD) nbDVDs++;
        }
        System.out.println(" - Livres: " + nbLivres);
        System.out.println(" - Magazines: " + nbMagazines);
        System.out.println(" - DVDs: " + nbDVDs);
    }

    private int compterEmpruntsEnCours() {
        int count = 0;
        for (Emprunt e : emprunts) {
            if (e.getDateRetourEffectif() == null) {
                count++;
            }
        }
        return count;
    }
}

```

2.7 Exercice 6 : Programme Principal (Main)

Objectif : Créer un programme qui teste toutes les fonctionnalités.

Solution :

```
public class Main {
    public static void main(String[] args) {
        // Créer la bibliothèque
        Bibliotheque biblio = new Bibliotheque("Bibliothèque
Centrale", "123 Rue de la Paix");

        // Créer des documents
        Livre livre1 = new Livre("L001", "1984", 1949,
                                "George Orwell", "978-0451524935",
328);
        Livre livre2 = new Livre("L002", "Le Petit Prince", 1943,
                                "Antoine de Saint-Exupéry", "978-
0156012195", 96);
        Magazine mag1 = new Magazine("M001", "Science & Vie", 2024,
1289, "Janvier");
        DVD dvd1 = new DVD("D001", "Inception", 2010, "Christopher
Nolan", 148);

        // Ajouter au catalogue
        biblio.ajouterDocument(livre1);
        biblio.ajouterDocument(livre2);
        biblio.ajouterDocument(mag1);
        biblio.ajouterDocument(dvd1);

        // Créer des membres
        Membre membre1 = new Membre("M001", "Dupont", "Jean",
                                "jean.dupont@email.com",
"0612345678");
        Membre membre2 = new Membre("M002", "Martin", "Sophie",
                                "sophie.martin@email.com",
"0698765432");

        biblio.ajouterMembre(membre1);
        biblio.ajouterMembre(membre2);

        // Test du polymorphisme
        System.out.println("\n=== TEST DU POLYMORPHISME ===");
        livre1.afficherInfos();
        System.out.println();
        mag1.afficherInfos();
        System.out.println();
        dvd1.afficherInfos();
    }
}
```

```
// Emprunter des documents
System.out.println("\n=== EMPRUNTS ===");
biblio.emprunterDocument("M001", "L001");
biblio.emprunterDocument("M001", "D001");
biblio.emprunterDocument("M002", "L002");

// Afficher les statistiques
biblio.afficherStatistiques();

// Afficher les informations d'un membre
System.out.println();
membre1.afficherInfos();

System.out.println("\n=== Programme terminé avec succès!
===");
    }
}
```

Partie 3 : Questions

1. Encapsulation

- a) Pourquoi déclare-t-on les attributs en private ?
- b) Quel est l'avantage d'utiliser des getters/setters plutôt que des attributs publics ?
- c) Donnez un exemple de validation qu'on pourrait ajouter dans un setter.

2. Héritage

- a) Qu'est-ce que le mot-clé 'extends' signifie ?
- b) À quoi sert 'super()' dans un constructeur ?
- c) Peut-on hériter de plusieurs classes en Java ? Pourquoi ?
- d) Quelle est la différence entre 'protected' et 'private' dans le contexte de l'héritage ?

3. Polymorphisme

- a) Qu'est-ce que l'annotation @Override signifie ?
- b) Expliquez comment le polymorphisme est utilisé dans la méthode afficherInfos().
- c) Pourquoi peut-on écrire : Document doc = new Livre(); ?
- d) Quelle méthode sera appelée si on écrit doc.getType() sur la ligne précédente ?

4. Abstraction

- a) Pourquoi la classe Document est-elle abstraite ?
- b) Peut-on créer une instance de Document directement ? Pourquoi ?
- c) Quelle est la différence entre une méthode abstraite et une méthode concrète ?
- d) Toutes les méthodes d'une classe abstraite doivent-elles être abstraites ?

5. Conception

- a) Pourquoi utilise-t-on ArrayList plutôt qu'un tableau classique ?
- b) Expliquez le rôle de la variable statique 'compteurId' dans la classe Emprunt.
- c) Pourquoi la classe Emprunt a-t-elle une référence vers Membre ET vers Document ?
- d) Comment pourriez-vous améliorer le système pour gérer plusieurs bibliothèques ?

Partie 4 : Exercices

Exercice 1 : Système de Réservation

Ajoutez un système de réservation pour les documents déjà empruntés :

- Créez une classe Reservation
- Un membre peut réserver un document emprunté
- Quand le document est retourné, le premier membre en liste d'attente est notifié

Exercice 2 : Catégories de Livres

Ajoutez un système de catégories pour les livres :

- Créez une énumération Genre (FICTION, NON_FICTION, SCIENCE_FICTION, etc.)
- Ajoutez un attribut genre à la classe Livre
- Créez une méthode rechercherParGenre() dans Bibliotheque

Exercice 3 : Interface Emprutable

Créez une interface pour rendre le système plus flexible :

```
public interface Emprutable {  
    boolean peutEtreEmprunte();  
    void marquerCommeEmprunte();  
    void marquerCommeDisponible();  
    int getDureeEmpruntMax();  
}
```

Modifiez vos classes pour implémenter cette interface.

Exercice 4 : Système de Notation

Ajoutez un système de notation des documents :

- Les membres peuvent noter les documents de 1 à 5 étoiles
- Calculez la note moyenne de chaque document
- Affichez les documents les mieux notés

Exercice 5 : Persistance des Données

Sauvegardez les données dans des fichiers :

- Créez des méthodes pour sauvegarder le catalogue dans un fichier texte
- Créez des méthodes pour charger les données au démarrage
- Utilisez le format CSV ou JSON

Bon courage!